
Telegraph English: Semantic Prompt Compression via Structured Symbolic Rewriting

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We introduce Telegraph English (TE), a prompt-compression protocol that rewrites
2 natural language into a symbol-rich, formally-structured dialect. Where token-
3 deletion methods such as LLMingua-2 train a classifier to delete low-importance
4 tokens at a fixed ratio, TE performs a full semantic rewrite: it decomposes the input
5 into atomic fact lines, substitutes verbose phrases with ~ 40 logical and relational
6 symbols, and lets the compression ratio adapt to each document’s information
7 density. A consequence of the line-structure rule is that compression and semantic
8 chunking become the same operation—each output line is an independently ad-
9 dressable fact, so the compressed representation is simultaneously a semantic index.
10 We evaluate TE on 4,081 question-answer pairs from LongBench-v2 across five
11 OpenAI models and two difficulty levels. At roughly 50% token reduction, TE pre-
12 serves 99.1% accuracy on key facts with GPT-4.1 and outperforms LLMingua-2
13 at matched compression ratios on every model and task tested. The gap widens on
14 smaller models—up to 11 percentage points on fine-detail tasks—suggesting that
15 explicit relational structure compensates for limited model capacity. We release
16 the grammar specification, compression prompt, benchmark data, and reference
17 implementation.

18 1 Introduction

19 Large language models are increasingly embedded in retrieval-augmented generation (RAG), multi-
20 agent orchestration, and long-context reasoning pipelines. Input cost scales linearly with token count,
21 so prompt compression—feeding fewer tokens to the model while preserving the information it
22 needs—has become a practical lever for controlling latency and cost.

23 Two families of approach exist. *Extractive* methods select a subset of tokens or sentences from the
24 input [Jiang et al., 2023, Pan et al., 2024]; *abstractive* methods paraphrase or summarise it [Chevalier
25 et al., 2023]. LLMingua-2 [Pan et al., 2024], currently the strongest published baseline, trains a
26 GPT-4-distilled XLM-RoBERTa-large classifier to delete tokens below an importance threshold at a
27 user-specified ratio.

28 Token deletion works, but it has structural limits that become visible once one looks past the
29 compression ratio. The ratio is fixed regardless of input density. Deleting tokens can sever co-
30 reference chains and destroy logical connectives, leaving the downstream model to hallucinate the
31 relationships between surviving fragments. Token-deletion methods are input-only preprocessors—
32 they compress the initial prompt, but generated output passes uncompressed to the next pipeline stage,
33 so multi-step agent systems cannot compound the savings. Most consequentially, token deletion
34 produces no structure: the output is a degraded copy of the input, unable to be indexed, selectively
35 pruned, or dynamically updated.

36 We propose Telegraph English (TE), a different kind of compression. Rather than selecting which
37 tokens to keep, TE rewrites the passage into a compact, formally-structured dialect. The original
38 sentence

39 *“According to research by Author A and colleagues (2023), the application of machine*
40 *learning techniques to medical diagnostics resulted in a 27.5% increase in early detection*
41 *rates while simultaneously reducing false positives by approximately 12% compared to*
42 *traditional methods.”*

43 becomes, under TE:

44 `ML→MEDICAL-DIAGNOSTICS: EARLY-DETECTION+27.5% ^ FALSE-POSITIVE-12%`
45 `[AUTHOR-A:2023]`

46 Sixty-eight tokens become fourteen. The causal relationship, both quantitative claims, and the citation
47 are each on record as separate, addressable units—and the phrase “application of...resulted in” has
48 collapsed into a single symbol.

49 What makes TE architecturally distinctive is a property that emerges from the grammar’s line-structure
50 rule: compression and semantic chunking are the same operation. Every TE output line contains
51 exactly one atomic fact—one claim, one relationship, one datum. This is not a post-processing step
52 but a consequence of how the grammar defines a legal output. The result is a representation that is
53 simultaneously compressed, retrieval-ready, and amenable to dynamic management: atomic lines are
54 individually embeddable; tagged sections support hierarchical context budgeting; and facts can be
55 updated, merged, or pruned without re-running the compressor.

56 **Contributions.** (1) A formal grammar specification for structured prompt compression (§3). (2) A
57 unified compression-and-chunking framework where semantic compression, retrieval-ready indexing,
58 and dynamic context management emerge from a single rewriting pass (§3, Appendix A). (3) A
59 large-scale empirical comparison against LLMingua-2 on 4,081 key-fact and 801 fine-detail QA
60 pairs across five models (§5). (4) Evidence that the advantage of semantic rewriting over token
61 deletion grows on smaller models and on detail-intensive tasks (§6). (5) A reference implementation
62 with CLI tools for compression, benchmarking, and error analysis.

63 2 Related Work

64 **Prompt compression.** LLMingua [Jiang et al., 2023] introduced budget-constrained prompt
65 compression using perplexity-based token selection. LLMingua-2 [Pan et al., 2024] improved on
66 this with a data-distillation approach: GPT-4 labels token importance on the MeetingBank corpus,
67 and an XLM-RoBERTa-large classifier learns to predict which tokens to delete. The architectural
68 constraint is that the output remains a degraded subset of the input tokens—no new structure is
69 introduced.

70 **Abstractive compression.** AutoCompressors [Chevalier et al., 2023] train summary tokens that
71 substitute for long contexts; RECOMP [Xu et al., 2024] compresses retrieved documents into
72 extractive or abstractive summaries tailored to the downstream task. Both are effective but lossy by
73 design—they discard information that cannot be recovered, and neither produces a structured output
74 that supports selective manipulation.

75 **Structured representations and agent context.** Chain-of-thought prompting [Wei et al., 2022]
76 demonstrates that imposing structure on LLM inputs improves reasoning. TE extends this insight
77 to compression: explicit logical and relational operators help downstream models reconstruct the
78 intended meaning more reliably than degraded natural language. For long-running agents, MemGPT
79 [Packer et al., 2023] addresses context-window growth via virtual memory hierarchies, and Reflexion
80 [Shinn et al., 2023] maintains explicit self-reflection buffers; both operate on natural-language
81 representations. TE offers a complementary strategy of structured, fact-level representations that can
82 be selectively updated and pruned without further LLM calls.

83 **Semantic chunking for RAG.** Standard RAG pipelines split documents using fixed token windows
84 or sentence-boundary heuristics [Lewis et al., 2020, Gao et al., 2023]. TE sidesteps the chunking

question: compression produces atomic fact lines as a structural by-product, so no separate chunking stage is needed. Conceptually adjacent are controlled natural languages such as Attempto Controlled English [Fuchs et al., 2008], but those are designed for knowledge representation and formal reasoning rather than compression, and consumed by formal reasoners rather than LLMs.

3 The Telegraph English Grammar

The grammar (version 5) lives in a 430-line specification document that doubles as the system prompt for the LLM-based compressor. We summarise its key design principles here; the full specification is supplementary material.

3.1 Foundations

Four principles govern the grammar, in strict priority order: (i) **fidelity over brevity**—no information may be dropped unless inferable from what remains; (ii) **atomic line structure**—each line contains exactly one claim, step, event, or question; (iii) **upper-case default**, except where case carries information (proper names, code, SI symbols); (iv) **target compression** $\sim 5\times$ when feasible, but correctness, auditability, and reversibility take strict priority over token reduction.

3.2 Symbol vocabulary

TE defines a fixed vocabulary of relational and logical operators. The full set numbers roughly 40; Table 1 shows the core symbols that appear in most compressions. Each symbol has a single, non-interchangeable meaning. The grammar caps symbol density at three consecutive symbols per line—a readability constraint learned from early iterations where dense symbol chains became opaque even to GPT-4.

Table 1: Core relational and logical operators in the TE symbol vocabulary. The full vocabulary contains roughly 40 symbols organised by function (causal, logical, comparative, modal).

Symbol	Meaning	Example
=	Definition / equality	VELOCITY=DISTANCE/TIME
→	Causation / flow	HEAT→EXPANSION
⇒	Logical implication	RAIN⇒WETNESS
∴	Therefore / conclusion	X>Y ∧ Y>Z ∴ X>Z
∵	Because / reason	MOTOR-FAILURE ∵ OVERLOAD
↑/↓	Increase / decrease	TEMPERATURE↑
∧/∨/¬	And / or / not	A∧B, ¬EVIDENCE
≈/≠	Approximate / not equal	COST≈USD10M
VS	Contrast (never causal)	MODEL-A VS MODEL-B

3.3 Tags and domain conventions

Beyond the symbol vocabulary, the grammar provides a tagging system and a set of domain-specific formatting rules. Tags handle the framing that natural language carries through verbose syntactic constructions: temporal state (PAST:, NOW:, FUTURE:), modality (LIKELY:, POSSIBLE:, CONF=0.87), roles (AGENT:, PATIENT:, INSTRUMENT:), scope (CTX: for shared context), and structured content types (DEF:, Q:/A:). Each tag does double duty: it collapses verbose framing into a single token and provides the structural handle that downstream systems use for selective retrieval and context management.

Domain conventions standardise the surface forms that vary most across writers: quantities (VAR=VALUEUNIT), citations ([AUTH:YEAR], DOI:, ARXIV:), financial data (USD10.5M, Y/Y+5%, +2.5PT), and URLs. Locking these down at the grammar level removes a class of factual-error failure modes the compressor would otherwise need to handle case by case.

117 3.4 Compressor self-verification

118 Two mechanisms keep the compressor honest within a single LLM call: a quality gate and a prescribed
119 distillation sequence. The quality gate is a 12-point checklist covering formatting consistency, symbol
120 precision, abbreviation policy, number formatting, information preservation, and citation integrity.
121 It is embedded directly in the compression prompt, so the compressor self-verifies output before
122 returning it. The distillation sequence prescribes a six-pass reasoning order: (1) concept identification,
123 (2) claim extraction, (3) relation mapping, (4) redundancy elimination, (5) numerical verification,
124 (6) citation cross-checking. This is a chain-of-thought scaffold inside a single inference, not a
125 multi-call pipeline. Ordering matters: numerical verification before citation cross-checking, because
126 citations sometimes attach to numerical claims that must be confirmed first.

127 3.5 Compression as semantic chunking

128 Compression and semantic chunking are not separate stages—they are the same operation. Every
129 TE output line is an atomic fact, every section is tagged, and every CTX: block defines a scope. The
130 structure falls out of the grammar’s line-structure rule, not from any additional processing, and it
131 enables three things that token-deleted text cannot support: **selective retrieval** (a query about adverse
132 events retrieves exactly the relevant line and its scope, no chunking heuristic required); **graduated**
133 **compression-on-read** (a context-assembly system can keep the most relevant lines at full fidelity,
134 retain only heading tags for moderately relevant sections, and drop irrelevant sections entirely—no
135 LLM call needed); and **continuous state refinement** (facts can be updated in-place, merged, or
136 pruned during a session). We call this the *compress-once, manage-continuously* principle. A worked
137 example and a more detailed treatment appear in Appendix A.

138 4 Experimental Setup

139 4.1 Dataset

140 LongBench-v2 [Bai et al., 2025] contributes 503 long-context multiple-choice items; we use their
141 source documents as our corpus, generating our own QA pairs per chunk (§4) rather than reusing
142 the bundled questions. We filter to three categories suitable for factual QA—Single-Document QA,
143 Multi-Document QA, and Long-Dialogue History Understanding—which leaves 339 documents.
144 NLTK sentence tokenisation chunks each one into segments of at most 1,000 words, producing
145 4,081 chunk-level evaluation units. The categories span technical reports, multi-source narrative
146 synthesis, and conversational history—three regimes where compression methods fail differently.
147 The 1,000-word chunk cap matches the practical input size for which prompt compression actually
148 saves money.

149 4.2 Compression

150 Each chunk is compressed into TE using the v5 grammar prompt with OpenAI’s o4-mini model.
151 Token counts are measured with tiktoken (cl100k_base). The mean compression ratio is 0.585—a
152 41.5% token reduction—with a range from 0.13 to 1.57. The upper end deserves explanation: rare,
153 very short inputs that are already informationally dense occasionally *expand* under TE, because the
154 grammar’s fidelity-first principle prohibits dropping information even when doing so would reduce
155 token count. This is a feature, not a failure. The full distribution is shown in Figures 1 and 2.

156 For the LLMLingua-2 baseline, the same chunks are compressed using the publicly available
157 llmlingua package at two retention rates: 0.50 (50% kept) and 0.33 (33% kept).

158 4.3 QA evaluation protocol

159 We design a multiple-choice protocol that isolates comprehension: can a model answer a factual
160 question correctly when reading compressed text instead of the original? GPT-4.1 generates a verbatim
161 QA pair from the original chunk, plus a semantically equivalent “modified answer” that prevents
162 simple string matching from inflating scores. GPT-4.1 (temperature 0.7) generates three plausible
163 distractors matched in style, length, and specificity. The modified answer and three distractors are
164 shuffled into a four-option question. The evaluation model sees the original, then the compressed text,

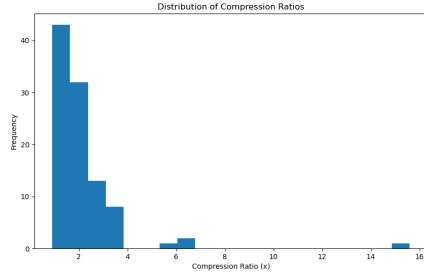


Figure 1: Distribution of compression ratios across 4,081 LongBench-v2 chunks compressed with TE (o4-mini, tiktoken cl100k_base). Mean 0.585, range 0.13–1.57. The right tail above 1.0 corresponds to short, dense inputs that expand under TE.

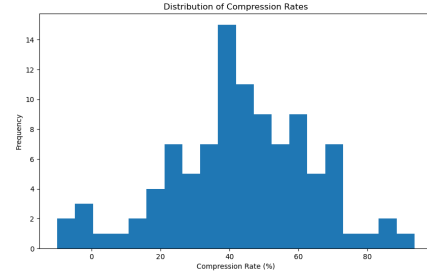


Figure 2: Distribution of per-chunk compression rate ($1 - \text{ratio}$) over the same corpus. The median chunk loses roughly 43% of its tokens; the bottom decile loses very little, reflecting TE’s adaptive behaviour on already-dense inputs.

165 and selects an answer in each setting. Accuracy is the fraction of correct selections; an *error* is a case
 166 where the model answered correctly on the original but incorrectly on the compressed version.

167 4.4 Test suites and models

168 Two suites probe different levels of information preservation. **key_facts** (4,081 QA pairs) targets core
 169 concepts—headline findings, main claims, central arguments—with generically plausible distractors.
 170 **fine_facts** (801 QA pairs) is adversarially designed to target information that lossy compression is
 171 most likely to destroy: precise numerical qualifiers, conditional statements, boundary conditions,
 172 secondary details. Distractors are near-miss variants—e.g. changing 4.8% to 4.3%—that can only be
 173 distinguished with access to the exact original detail.

174 We evaluate five OpenAI models: GPT-4.1, GPT-4o, GPT-4o-mini, GPT-4.1-nano, and a fine-tuned
 175 GPT-4o variant. GPT-4.1 also generates the QA pairs and distractors. Different suites use different
 176 model subsets: **key_facts** is run on GPT-4.1, GPT-4o-mini, and GPT-4.1-nano; **fine_facts** on GPT-4o
 177 and GPT-4o-mini. The fine-tuned variant is not used as a separate accuracy benchmark—it serves as
 178 a sanity check that fine-tuning on the original distribution does not change comparative behaviour at
 179 compression-decoded inputs.

180 5 Results

181 5.1 Key facts accuracy

Table 2: Accuracy on the **key_facts** suite (4,081 QA pairs). TE is Telegraph English at ~50% compression; LLML2-50 is LLMLingua-2 at 50% retention. Drop is in percentage points (pp) relative to original. **Bold** = best compressed.

Model	Original	TE	LLML2-50	TE Drop	LLML2-50 Drop
GPT-4.1	1.000	0.991	0.990	−0.9	−1.0
GPT-4o-mini	0.991	0.957	0.946	−3.4	−4.5
GPT-4.1-nano	0.980	0.950	0.949	−3.0	−3.1

182 On headline facts (Table 2), TE matches or edges out LLMLingua-2 across the board. The accuracy
 183 loss is negligible for GPT-4.1—less than a percentage point while halving the token count. The gap
 184 widens on smaller models: 1.1 pp on GPT-4o-mini, with the same direction at GPT-4.1-nano. Not
 185 dramatic. But consistent—the direction never reverses across configurations.

Table 3: Accuracy on the adversarial `fine_facts` suite (801 QA pairs). Fine-detail tasks expose larger compression effects; TE preserves more than LLMLingua-2 at matched retention.

Model	Original	TE	LLML2-50	TE Drop	LLML2-50 Drop
GPT-4o	0.996	0.965	0.933	−3.1	−6.3
GPT-4o-mini	0.938	0.843	0.820	−9.5	−11.8

186 5.2 Fine facts accuracy

187 Fine details are harder (Table 3). Compression loss runs 3–4 $\times$ higher than on key facts, regardless of
 188 method. TE holds an advantage of 3.2 pp over LLMLingua-2 on GPT-4o and 2.3 pp on GPT-4o-mini
 189 at matched 50% retention. The fine-facts regime is where token deletion starts to break down: the
 190 very tokens the questions probe—numerical qualifiers, conditional caveats, secondary details—are
 191 precisely the ones an importance classifier flags as low-value.

192 5.3 Accuracy hierarchy and compression statistics

193 Across all models and tasks the ranking holds without exception: original > TE > LLML2-50 >
 194 LLML2-33. TE’s mean compression ratio of 0.585 (std = 0.254) hides a wide spread: half the corpus
 195 sits between 0.41 and 0.74, with median 0.57. Documents dense with technical content or data tables
 196 resist compression; verbose narrative text yields ratios of 5:1 or better. Fidelity-first design means the
 197 ratio is an outcome, not a parameter.

198 5.4 Error analysis

199 Of the 4,081 `key_facts` items, 187 (4.6%) were correct on the original and incorrect on TE for
 200 GPT-4.1-nano. These error cases have a mean compression ratio of 0.531, slightly more compressed
 201 than the population mean—aggressive compression and error risk are correlated. Failures cluster
 202 around fine details: dates, units, conditional qualifications, and numerical relationships where TE
 203 either abbreviates a critical modifier or collapses a distinction the question specifically probes. One
 204 characteristic failure: a legal-document chunk where TE compressed “no later than 30 calendar days
 205 after receipt of written notice” into DEADLINE=30D-AFTER-NOTICE, and the question asked whether
 206 the deadline was in calendar or business days. The 30D abbreviation does not distinguish. This is a
 207 limitation of the symbol vocabulary, not a compressor error.

208 6 Analysis

209 6.1 Why semantic rewriting outperforms token deletion

210 Four mechanisms explain the pattern in the results. They are not ranked; different mechanisms
 211 dominate in different regimes. **Semantic-unit preservation**: token deletion operates at the token
 212 level and can split multi-word expressions, sever noun-modifier pairs, strand a number from its unit;
 213 TE works one level up, with related concepts grouped into hyphenated compounds and complete
 214 claims occupying single lines. **Explicit logical structure**: when LLMLingua-2 deletes a connective
 215 like “therefore” or “in contrast to,” the downstream model has to guess the relationship; TE refuses to
 216 offer the guess, with `∴`, `VS`, `→` each unambiguous and preserved regardless of what else is removed.
 217 **Co-reference stability**: TE’s one-claim-per-line discipline and upper-case entity naming eliminate
 218 pronoun resolution ambiguity; token deletion can strand a pronoun whose antecedent has been
 219 removed. **Adaptive compression**: a fixed-ratio method compresses dense and verbose passages
 220 identically; TE does not—dense passages emerge at ratios near 1.0, verbose ones below 0.2. The four
 221 mechanisms interlock, which is why LLMLingua-2 cannot match TE by adopting any single one of
 222 them.

223 6.2 The small-model effect

224 The TE advantage grows as model capacity shrinks. GPT-4.1 barely notices the difference between
 225 TE and LLMLingua-2 on key facts; GPT-4.1-nano and GPT-4o-mini show a wider gap, and on fine

facts the divergence becomes substantial. The likely explanation is capacity-dependent. Smaller models have less ability to reconstruct implicit relationships from token-deleted fragments; TE compensates by offloading that reconstruction work to the compression stage—the evaluation model receives a representation where the relationships are already marked, rather than having to hallucinate them from sparse clues. This has practical weight: smaller models are precisely the ones deployed in cost-sensitive production pipelines, which is where prompt compression earns its keep.

6.3 The fine-facts gap

Key facts survive both compression methods reasonably well. Central claims are often redundantly signalled, and even aggressive token deletion tends to preserve them. Fine details are stubborn in a different way: precise numerical qualifiers, conditional caveats, and secondary attributions are exactly the tokens an entropy-based classifier flags as low-importance in isolation. A number like “4.8%” may look dispensable next to surrounding prose. But if the question asks whether the figure was 4.8% or 4.3%, that token is the entire answer. TE’s claim-level decomposition and explicit numerical formatting (+27.5%, CONF=0.87, Y/Y+12.3%) are designed to preserve these details: numbers are never abbreviated, always attached to their units, and always placed in a structured format the downstream model can parse unambiguously.

6.4 Pipeline-level cost

There is a structural difference between the two methods that the accuracy comparison alone obscures: LLMingua-2 operates as an input-only preprocessor. It compresses the initial prompt; generated output passes uncompressed to subsequent stages. TE can persist as a native format throughout a pipeline. Consider a five-step agent pipeline with 2,000 tokens of initial context and five generation steps averaging 400 tokens each, at \$10 per million tokens (Table 4). The savings compound because each stage operates on TE-formatted text. LLMingua-2 compresses only the first stage’s input; the remaining four stages process uncompressed output at full token cost. A more architectural treatment of dynamic context management appears in Appendix B.

Table 4: Pipeline-level cost for a five-step agent pipeline (2,000-token initial context, five 400-token generations) at \$10 per million tokens. TE persists across stages; LLMingua-2 compresses only the first stage.

Method	Total tokens	Cost / 1K calls	Savings
Original	4,000	\$40	—
LLMingua-2	~3,300	\$33	\$7
Telegraph English	~1,600	\$16	\$24

7 Implementation

The reference implementation is a Python package with five pipeline stages: synchronous and asynchronous (Batch API) compression of LongBench-v2 documents using the TE grammar prompt; automated quality review via Claude (structured JSON scores 0–10 with strengths, weaknesses, and example pairs); end-to-end QA benchmarking (generation, distractor creation, MC evaluation); LLMingua-2 baseline evaluation against the same QA pairs; and error analysis with case-level output. Each stage is accessible as both a CLI command and an importable library function.

8 Limitations

LLM-dependent compression. TE requires an LLM call per chunk, adding latency and cost at compression time. This is amortised when compressed text is reused, but TE is poorly suited for compressing ephemeral inputs that will be read once and discarded. **Proprietary evaluation models.** Our benchmark relies on OpenAI models that are not open-weight, limiting reproducibility; future work should extend evaluation to open models. **English only.** The grammar and benchmarks are English; adapting the symbol vocabulary to other languages—particularly agglutinative or logographic ones—is non-trivial. **Compressor model sensitivity.** TE quality depends on the model performing

the rewrite; we have not yet mapped this sensitivity curve. **QA generation bias.** Both QA pairs and evaluations are produced by OpenAI models; an ideal evaluation would include human-written questions or a diverse set of QA generators. **Dynamic context management is not yet benchmarked.** The semantic chunking and dynamic state-management capabilities described in §3 and Appendix B are architectural arguments, not empirical results from a multi-turn evaluation. We have demonstrated format-level feasibility; we have not measured downstream effects over extended sessions. This is the most important gap in the current evaluation. **Comparison scope.** We benchmark against LLMingua-2 only—currently the strongest published baseline at our compression ratios. A broader comparison against AutoCompressors, RECOMP, and more recent methods would strengthen the claims.

9 Conclusion

Telegraph English demonstrates that structured semantic rewriting is a viable alternative to token deletion for prompt compression—and, on the evidence presented here, a better one. The advantage is largest where compression matters most practically: on smaller, cheaper models and on fine-grained details. The quantitative comparison may not be the most interesting part of this work. Token-deletion methods produce a smaller copy with no internal organisation; TE produces a representation where every line is an identified fact, every section is tagged, every relationship is marked with an explicit symbol. That structure makes the output not just smaller but more *useful*—more retrievable, more auditable, more maintainable over time. The compress-once, manage-continuously principle is, at this stage, an architectural argument rather than an empirical result; validating it in production agent systems is the obvious next step.

References

- Yushi Bai et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3639–3664, 2025.
- Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of EMNLP*, 2023.
- Norbert E. Fuchs, Kaarel Kaljurand, and Tobias Kuhn. Attempto controlled english for knowledge representation. In *Reasoning Web*, volume 5224 of *Lecture Notes in Computer Science*, pages 104–124. 2008.
- Yunfan Gao et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmilingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, 2023.
- Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of ACL*, 2024.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

313 Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with
 314 context compression and selective augmentation. In *International Conference on Learning Repre-*
 315 *sentations (ICLR)*, 2024.

316 A Compression as Semantic Chunking: Worked Example

317 To make §3 concrete, consider a multi-paragraph clinical-trial summary compressed into TE:

```

318 H1: CLINICAL-TRIAL OUTCOMES
319 CTX: PHASE-III RANDOMISED CONTROLLED-TRIAL(RCT); N=2400
320 PRIMARY-ENDPOINT: MORTALITY↓23% VS PLACEBO; p<0.001
321 [AUTHOR-B:2024]
322 SECONDARY-ENDPOINT: HOSPITALIZATION↓18%; p=0.003
323 ADVERSE-EVENTS: NAUSEA=12% ^ HEADACHE=8% ^ SERIOUS=2.1%
324 H1: SUBGROUP-ANALYSIS
325 AGE>65: MORTALITY↓31% (STRONGER-EFFECT)
326 AGE<65: MORTALITY↓14% (WEAKER-EFFECT)
327 CONF=0.92 FOR INTERACTION-EFFECT
328 H1: LIMITATIONS
329 FOLLOW-UP=18 MONTHS; LONG-TERM-EFFECTS UNKNOWN
330 EXCLUSION: PATIENTS WITH RENAL-IMPAIRMENT

```

331 Each line is a fact; each heading is a section boundary; each CTX: block defines a scope. The structure
 332 falls out of the grammar’s line-structure rule rather than from any additional processing, and it enables
 333 three things that token-deleted text cannot support.

334 **Selective retrieval.** A query about adverse events retrieves exactly the ADVERSE-EVENTS line and
 335 its CTX: scope. No sliding-window heuristic, no overlap parameter, no risk of splitting a relevant fact
 336 across chunk boundaries; the semantic boundaries are intrinsic to the format.

337 **Graduated compression-on-read.** When assembling a prompt under a tight token budget, an agent
 338 can apply different policies to different sections: keep the lines most relevant to the current query
 339 at full fidelity; retain only the heading tags (H1: LIMITATIONS) for moderately relevant sections,
 340 preserving topic structure at near-zero cost; drop irrelevant sections entirely. This second-stage
 341 compression is semantically principled—it operates on identified sections, not on token positions.

342 **Continuous state refinement.** During a conversation, facts from earlier turns can be revised without
 343 re-compressing the source: *update* (replace a corrected figure in place), *merge* (combine related facts
 344 when the distinction no longer matters), *prune* (remove claims that have moved past relevance), and
 345 *promote/demote* (expand a heading-collapsed section, or collapse a fully expanded one).

346 B Beyond Static Compression: Dynamic Context Architecture

347 The results in §5 measure TE as a static compression method—compress once, read once, evaluate.
 348 This is the fair comparison against LLMLingua-2 and where the benchmark numbers live. But the
 349 more consequential property of TE may not be the compression ratio; it is the structure of the output.

350 **Unifying compression and chunking.** Conventional RAG systems run documents through two
 351 stages: chunking (splitting into fixed-size segments for embedding) and optional compression
 352 (reducing each chunk’s token count). These stages have different objectives and can interfere—a
 353 chunk boundary splits a sentence, then compression deletes the tokens needed to reconstruct it. TE
 354 collapses both stages into one. Each output line is a complete semantic unit; the chunking boundaries
 355 *are* the compression output. A TE-compressed document is immediately embeddable at the line
 356 level. The practical consequence for retrieval precision: fixed-window chunking inevitably includes
 357 irrelevant context within each chunk and risks splitting relevant information; TE surfaces exactly the
 358 facts a query matches, at the granularity of individual claims.

Hierarchical context budgeting. Because TE output is tagged with headings, context scopes, and role markers, a context-assembly system can make graduated decisions about inclusion. For a given token budget: full-fidelity inclusion of all atomic lines for the most relevant sections; heading-only retention for moderately relevant sections, preserving topic structure at near-zero cost; omission of irrelevant sections entirely. This graduated policy can achieve very high total compression (10–50 $\times$) when only a fraction of the document is relevant, while maintaining full detail where it matters. The policy operates on the TE output’s structure—no LLM call needed.

Dynamic state in agentic sessions. Long-running agent sessions accumulate context over many exchanges. The standard solutions are blunt: hard truncation drops the oldest tokens regardless of relevance; periodic summarisation requires an LLM call and is irreversible. TE enables something finer. Because context is already decomposed into tagged atomic facts, an agent can maintain a living state: *fact updates* replace the old line in place rather than appending alongside it; *redundancy pruning* removes facts whose information has been absorbed by later ones; *scope closure* collapses an entire CTX: block to a heading once a topic is resolved; *priority re-ranking* reorders facts by current relevance, placing the most important context where transformer attention is strongest. Context growth is controlled by continuously refining the active fact set, not by discarding the oldest tokens. This is cheap (string manipulation, no LLM calls) and semantically principled.

The cost profile is asymmetric by design: one expensive LLM rewrite per document, then indefinite cheap manipulation of the structured output.

C Full Results Tables

Table 5: Complete key_facts results with compression statistics.

Model	n	Original	TE	LLML2-50	TE Drop	LLML2-50 Drop	Mean ratio
GPT-4.1	4,081	1.000	0.991	0.990	−0.9	−1.0	0.585
GPT-4o-mini	4,081	0.991	0.957	0.946	−3.4	−4.5	0.585
GPT-4.1-nano	4,081	0.980	0.950	0.949	−3.0	−3.1	0.585

Table 6: Complete fine_facts results.

Model	n	Original	TE	LLML2-50	TE Drop	LLML2-50 Drop
GPT-4o	801	0.996	0.965	0.933	−3.1	−6.3
GPT-4o-mini	801	0.938	0.843	0.820	−9.5	−11.8

Table 7: Compression ratio statistics (tiktoken cl100k_base, $n = 4,081$ chunks).

Statistic	Value
Mean	0.585
Std	0.254
Min	0.000
25th percentile	0.407
Median	0.570
75th percentile	0.739
Max	1.567

Table 8: Error analysis: `key_facts` items correct on original, incorrect on TE (GPT-4.1-nano).

Statistic	Value
Total items	4,081
Error items	187
Error rate	4.58%
Mean compression ratio (errors)	0.531
Mean compression ratio (all)	0.585